

I have an idea in terms of at least getting our hands a bit dirty with this formulation, I want you to know something very important: 1- I have very surface and at levels zero understanding and knowledge of simplicial complexes sheafs and cohomology and I plan to learn along side this research project 2- look at proj III, to be able to have something motivational and actually come out with an interesting toy model to generalize, I suggest we design a specific model aimed to solve this paper, so we'll build our cognitive space around the ability to read this, search and learn then solve this mathematical problem. this way we'll fix and adjust the formulation according to what comes out of us. 3- I think making out with my girlfriend and completely giving up on this project is the best thing to do right now. she's so hot and wants to marry me.

README.md

File

read this, where this crazy adventure started is with wanting what ever I built be more fluid and dynamic, here's what I think would work as a week's project, we try to implement this formalism to fit the different parts of what was already built here, inside our simplicial complex, and let C be the complex with vertices each part of this brain, then we can have the sheaf come in the way it does, and we can try to build an adequate algebra that will truly turn this model into something that can surpass claude and gpt... do you think it's possible?

Monday 10:08 AM

COGNITIVE STATE SPACES(1).pdf

PDF

opinion?

I actually think this changes my advice quite a bit.

Initially, I kept trying to **derive the mathematics** because I thought we were still in the theory-building phase.

What you've just said is much closer to **software architecture** than to mathematical invention.

This sentence is the key:

"This is nothing more than a blueprint on how to fix an overarching bug pestering MOS right now."

That completely changes the evaluation criteria.

We're no longer asking:

"Is this the ultimate mathematical model of cognition?"

We're asking:

"Does this architecture eliminate the structural limitations of the current MOS while remaining compatible with everything already implemented?"

That's a much better engineering question.

I think we should treat the document like an RFC (Request for Comments)

Not a paper.

Not a manifesto.

An RFC.

Meaning:

Every statement must satisfy three questions.

1. Does it solve an existing problem?

For example,

Current MOS

Modules communicate through prompts.

Problem

State isn't explicit.

Solution

Introduce

$M=(C,F,s)$.

Excellent.

Current MOS

Memory retrieval is procedural.

Problem

No persistent geometric organization.

Solution

Dynamic simplicial complex.

Again,

excellent.

Notice

every new object exists because it fixes something.

Not because it's mathematically beautiful.

2. Does it preserve existing functionality?

This is where I think we need to be extremely disciplined.

Suppose MOS already has

- Planner
- Verifier
- Search
- Reflection
- Memory
- Sandbox

None of these should disappear.

Instead

they become

clients of

M.

They no longer own state.

They operate on state.

That is a software refactoring.

Not a redesign.

3. Can it be implemented immediately?

This is where I think we need a hard rule.

I would literally write this at the beginning of the project.

Every mathematical definition must correspond to an immediately implementable C++ abstraction.

Example

Mathematics

$\mathcal{C}$

↓

class SimplicialComplex

Mathematics

$\mathcal{F}$

↓

class Sheaf

Mathematics

$\mathcal{S}$

↓

class CognitiveState

Mathematics

D

↓

class CognitiveOperator

If we can't write the class,

the definition isn't mature enough.

I also agree with something you said that I hadn't fully appreciated.

You wrote

"debugging will go one on one with refining the ideas."

I actually think that's exactly right.

This is how numerical mathematics is often developed.

You don't first prove everything.

You implement.

You discover pathological cases.

Then the definitions improve.

The implementation becomes a mathematical experiment.

The thing I would absolutely forbid

This is where I think we need to protect ourselves.

No new mathematical objects unless implementation forces them.

Suppose during coding we realize

"Restriction maps aren't expressive enough."

Great.

Now we have evidence.

Generalize.

Suppose instead we say

"Maybe we should use ∞ -categories."

No.

Because nothing failed yet.

Implementation hasn't asked for them.

I think we should redefine the roadmap.

Instead of

Theory

↓

Implementation

I think it should be

Definition

↓

C++ implementation

↓

Visualization

↓

Experiment

↓

Debug

↓

Mathematical refinement

That cycle repeats forever.

That is exactly how computational mathematics is done.

Here's the most important thing I realized after reading your message.

I think we've been talking about

$$M = (C, F, s)$$

as if it were

the AI.

I no longer think that's true.

I think

M

is

the operating system.

Everything else already existing in MOS—

- Planner

- Search
- Reasoner
- Reflection
- Simulation
- Verification

are **processes**.

Processes don't own knowledge.

They operate on the operating system.

That is a profound architectural simplification.

Instead of every subsystem maintaining its own internal notion of state, there is one canonical cognitive state space. Every subsystem becomes an operator acting on that shared object.

This also answers something that worried me before.

I kept asking

"Where do the operators come from?"

Now I think I know.

They come from the modules already present.

Each subsystem already performs transformations.

Planner

↓

changes goals.

Memory

↓

stores.

Search

↓

expands.

Verification

↓

checks consistency.

Reflection

↓

compresses.

Those are already operators.

The algebra is simply

the algebra generated by these transformations,

after we've expressed them as actions on

M .

Notice how much more conservative this is.

We're not inventing cognition.

We're **lifting the existing MOS architecture into a common mathematical state space.**

This is why I think the project is now much more achievable.

The cognitive state space is no longer competing with MOS; it's **factoring MOS**. It provides a single, explicit mathematical substrate on which the existing components act. That means your implementation effort isn't "replace everything with sheaves and simplicial complexes." It's "refactor every subsystem so that it reads from and writes to the same geometric state."

If you keep that discipline, then every line of code you write next week directly improves the existing system rather than creating a parallel prototype. And because the mathematics and the implementation evolve together through debugging, the theory will become more precise without drifting beyond the scope you've already defined. I think that's exactly the right balance between ambition and engineering pragmatism.

not clients dude read the blueprints they're the nodes

You're right. I misunderstood the architectural role after reading your blueprint.

I was still thinking in terms of a traditional software architecture where modules operate on a shared state. **Your blueprint is doing something much more radical.**

If I understand it correctly now, the picture is closer to this:

$$M = (C, F, s)$$

C

Planner ----- Memory

| \ / |

| \ / |

| \ / |

Search--Reasoning--Verification

\ | /

\ | /

Reflection

These are **not external agents** acting on M .

They **are the vertices of C** .

That changes everything.

The modules ARE the geometry

This is actually much more interesting.

Instead of saying

Planner modifies memory

you're saying

Planner is a **local region** of the cognitive manifold.

Its neighborhood determines

- who it communicates with,
- what information it shares,
- how reasoning propagates.

Now the simplicial complex isn't representing concepts.

It's representing **the cognitive architecture itself**.

That is a very different interpretation.

Then the sheaf becomes almost inevitable

Each vertex possesses a local state.

For example

Planner

stores

- current objective
- subgoals
- priority

Memory stores

- retrieved concepts
- embeddings
- local cache

Verification stores

- proof obligations
- confidence
- contradictions

Reasoning stores

- deductions

- hypotheses

Each of these is a stalk.

Then restriction maps explain

how information flows between neighboring cognitive regions.

That is actually a very natural sheaf interpretation.

And THIS is where I think the operator algebra belongs

This was the piece I couldn't place before.

The operators do **not** operate on software modules.

The operators operate on the **geometric organization of cognition**.

Examples

Focus

↓

changes activation

S

Expand

↓

creates new simplices

inside

C

Integrate

↓

changes restriction maps

inside

F

Compress

↓

contracts regions of the complex

Restructure

↓

changes the topology itself.

Now every operator has a geometric target.

The really beautiful consequence

Suppose the Search and Reasoning modules repeatedly cooperate.

Instead of just passing messages,

the topology itself changes.

Eventually

Search

Reasoning

Verification

become a 2-simplex.

Meaning

they've become a higher-order cognitive unit.

That's an incredibly elegant interpretation of learning.

Learning isn't merely changing parameters.

Learning literally changes the topology.

The only thing I would still question

This is the one thing I would challenge before implementing.

You're currently using the vertices as **functional subsystems**.

But eventually you'll also want the system to represent **knowledge**.

Where does that knowledge live?

I now see two possibilities.

Option A

Knowledge lives inside the stalks.

Vertices remain cognitive modules.

This keeps C as the architecture graph.

Option B

Knowledge itself also becomes simplicial.

Now C mixes

- modules
- concepts

I think this becomes messy.

I would strongly recommend Option A

Keep

C

representing

the brain.

Keep

F

representing

everything the brain currently knows locally.

Then the sheaf is literally the distributed knowledge across the cognitive architecture.

That separation is very clean.

One thing I think your blueprint hints at—but doesn't quite say

If the vertices are the subsystems, then $\mathcal{C}$ **should not be static.**

That's where your original motivation ("fluidity") finally reappears.

Initially, the graph might look like this:

Planner --- Reasoner --- Memory

After solving a difficult mathematical problem, perhaps Reflection and Verification become much more tightly coupled, so the topology evolves:

Planner --- Reasoner --- Memory

\ | /

\ | /

Reflection---Verification

Or even new higher-dimensional simplices appear to represent stable cooperative behaviors.

This means the *architecture itself learns*, not just the information stored within it.

I think this is the key insight your blueprint is converging toward:

- **Vertices are not data. They are cognitive processes.**
- **The sheaf is not the architecture. It is the distributed local knowledge attached to that architecture.**
- **The cognitive state is the current coherent activation of the entire system.**
- **The operator algebra governs how both the architecture and its distributed knowledge evolve over time.**

That interpretation is internally consistent, preserves your existing MOS design philosophy, and gives every piece of the mathematics a concrete computational role. It's also much closer to a true **cognitive operating system** than to a knowledge graph, because the geometry represents the organization of computation itself rather than just the organization of facts.

I want Claude to study how implementable this paper is to my current project... I'll give it the current github link to the project and this PDF and ask it how doable is implementing this idea and does it actually solve the problem with MOS, write me the best prompt for Claude.

I would not ask Claude whether the idea is "good." That's too subjective and will often produce encouragement rather than engineering feedback.

Instead, I'd ask it to act as a **principal software architect and skeptical reviewer** whose job is to determine whether your proposed mathematical formalism is actually compatible with the existing MOS codebase.

Here's the prompt I would use.

Prompt

You are acting as a **Principal Software Architect, AI Research Engineer, Computational Mathematician, and Skeptical Reviewer**.

I am going to provide you with two things:

1. The complete GitHub repository of my current project (MOS).
2. A research blueprint describing a proposed mathematical reformulation of its internal cognitive architecture.

Your job is **NOT** to tell me whether the ideas are interesting.

Your job is to determine, with maximum technical honesty, whether this blueprint is actually implementable inside the existing project.

Assume I have no interest in rewriting MOS from scratch.

The proposed mathematical framework must integrate into the current architecture while preserving the already implemented functionality.

Treat this as an architectural review before beginning a major refactor.

OBJECTIVE

Determine whether the proposed mathematical framework is capable of fixing the current structural limitations of MOS without breaking the existing architecture.

I want a brutally honest engineering assessment.

Do not optimize for optimism.

Optimize for correctness.

Whenever you criticize something, propose a concrete alternative.

IMPORTANT CONTEXT

The PDF is **NOT** intended to invent a new branch of mathematics.

It is a blueprint for refactoring the internal representation of cognition inside MOS.

The goal is to replace the current implicit cognitive state with an explicit mathematical state space

$$M = (C, \mathcal{F}, s)$$

where

- C is a simplicial complex representing the existing cognitive architecture,
- each vertex of C corresponds to one subsystem already present inside MOS (Planner, Memory, Search, Reflection, Verification, Sandbox, etc.),
- the sheaf stores the local information available to each subsystem,
- the global section represents the coherent cognitive state,
- an operator algebra evolves the geometry itself.

Nothing in the blueprint is intended to replace the modules.

The modules already exist.

The blueprint proposes a mathematical state space that organizes them.

This is fundamentally a software architecture refactor rather than a complete redesign.

TASKS

Please perform the following review.

Part I — Reverse Engineer MOS

Study the GitHub repository carefully.

Produce a complete architectural analysis.

Identify:

- all major modules
- dependencies
- information flow
- memory management
- planning
- reasoning
- retrieval
- verification
- orchestration
- tool usage
- bottlenecks
- current cognitive state representation

Explain how the system currently works.

Part II — Compare MOS to the Blueprint

Read the PDF in detail.

For every major concept introduced in the paper, determine

- whether it already exists inside MOS,

- whether it replaces an existing component,
- whether it augments an existing component,
- whether it conflicts with the existing architecture,
- whether it is unnecessary.

Produce a detailed comparison table.

Part III — Feasibility Analysis

For every proposed mathematical object, evaluate implementation feasibility.

Examples include:

- simplicial complex
- sheaf
- stalks
- restriction maps
- global sections
- cognitive state
- operator algebra
- topology evolution
- dynamic restructuring

For each one discuss:

- implementation complexity
- computational cost
- memory requirements
- integration difficulty
- runtime implications
- scalability

Rate each component

- Easy
- Moderate
- Difficult
- Extremely Difficult

and justify every rating.

Part IV — Software Architecture

Assuming the blueprint is adopted,
design the software architecture.

Do NOT redesign MOS.

Instead show how each existing module would be refactored to operate within

$$M = (C, \mathcal{F}, s).$$

For every current subsystem explain

- where it lives inside the simplicial complex,
 - what information belongs inside its stalk,
 - what operators it naturally induces,
 - how it communicates with neighboring vertices.
-

Part V — Evaluate the Mathematics

Critically evaluate whether the mathematics is actually appropriate.

Specifically assess:

- simplicial complexes
- sheaf theory
- cohomology
- operator algebras

- D-module inspiration
- topology evolution

Determine whether any of these are

- mathematically unnecessary,
- computationally unjustified,
- excessively complicated,
- or missing important supporting theory.

If a simpler existing mathematical framework would achieve the same objective, explain why.

Part VI — Does This Actually Fix MOS?

This is the most important section.

Identify the current weaknesses of MOS.

For each weakness determine whether the proposed architecture genuinely fixes it.

Do not assume it does.

Provide evidence from both the codebase and the mathematical blueprint.

If it does not solve a problem, explain why.

If it introduces new problems, explain those as well.

Part VII — Refactoring Strategy

Produce a concrete implementation roadmap.

The roadmap must preserve the current codebase.

Every step should be incremental.

Avoid rewriting working components.

For every phase specify

- files affected

- new classes
 - interfaces
 - migration strategy
 - testing strategy
 - expected risks
-

Part VIII — Missing Pieces

Identify everything the blueprint is missing before implementation can begin.

For example:

- undefined mathematical objects
- ambiguous interfaces
- missing algorithms
- incomplete definitions
- hidden assumptions
- unresolved edge cases
- impossible implementation details

These should become the next research tasks.

Part IX — Final Verdict

Answer these questions explicitly.

1. Is this blueprint implementable inside the current MOS architecture?
2. Does it require a full rewrite or only a refactor?
3. Which ideas are immediately implementable?
4. Which ideas require significant mathematical development first?
5. Which ideas should be postponed?
6. Which ideas are likely to fail in practice?

7. Does this architecture genuinely improve MOS or merely make it mathematically more complicated?
 8. If implemented successfully, what concrete capabilities would MOS gain that it currently lacks?
 9. What is the biggest technical risk?
 10. If you were leading this project, what would you change before writing the first line of code?
-

REVIEW STYLE

Do not be polite.

Do not be optimistic.

Do not speculate.

Think like a senior engineer responsible for maintaining a production codebase and a reviewer deciding whether this architecture is mature enough to begin implementation.

Whenever possible, reference specific files, classes, and functions from the repository.

Ground every conclusion in evidence from either the codebase or the blueprint.

One suggestion

I would ask Claude for **one more thing**, because I think it will be incredibly valuable:

"Assume you are joining this project as its lead software architect. Your primary responsibility is to preserve the strengths of the existing MOS while integrating the proposed geometric cognitive state with the fewest possible architectural changes. At every step, prefer adaptation over replacement. If any proposed mathematical object cannot be implemented cleanly within the existing codebase, explain precisely why and propose the smallest mathematically consistent modification that would make it implementable."

That addition shifts the review from "Is this idea cool?" to the question you actually care about: **"Can this blueprint become the next evolution of MOS without destroying everything that's already working?"** That's the kind of critique most likely to uncover real implementation issues before you spend weeks coding.

